

Exploration

Module 2 Hands-on

MD01 - Shaquille Nathan Kalevi

Google Developer Groups on Campus - ITB

To-Do:

Expected Output

1. Screenshot of the running application. ⌚
2. Screenshot of the code structure. ⌚
3. Link to a public GitHub repository. ⌚

Bonus Tasks

Bonus 1: Theme Toggle ⌚

Bonus 2: Dynamic Skill List with Add Button ⌚

Bonus 3: Async "Connect" Button with Loading State ⌚

1. Make new project

Here is what I am doing to initialize the project:

- I open my terminal and run the command `flutter create gdgoc_profile_card`.
- I wait for the Flutter SDK to generate the boilerplate code.
- Then, I open the project directory in VS Code to start coding.

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\>flutter create gdgoc_profile_card

A new version of Flutter is available!
To update to the latest version, run "flutter upgrade".

Creating project C:\gdgoc_profile_card...
Resolving dependencies in 'C:\gdgoc_profile_card'... (1.4s)
Downloading packages...
Got dependencies in 'C:\gdgoc_profile_card'.
Wrote 131 files.

All done!
You can find general documentation for Flutter at: https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/flutterdev

In order to run your application, type:

$ cd C:\gdgoc_profile_card
$ flutter run

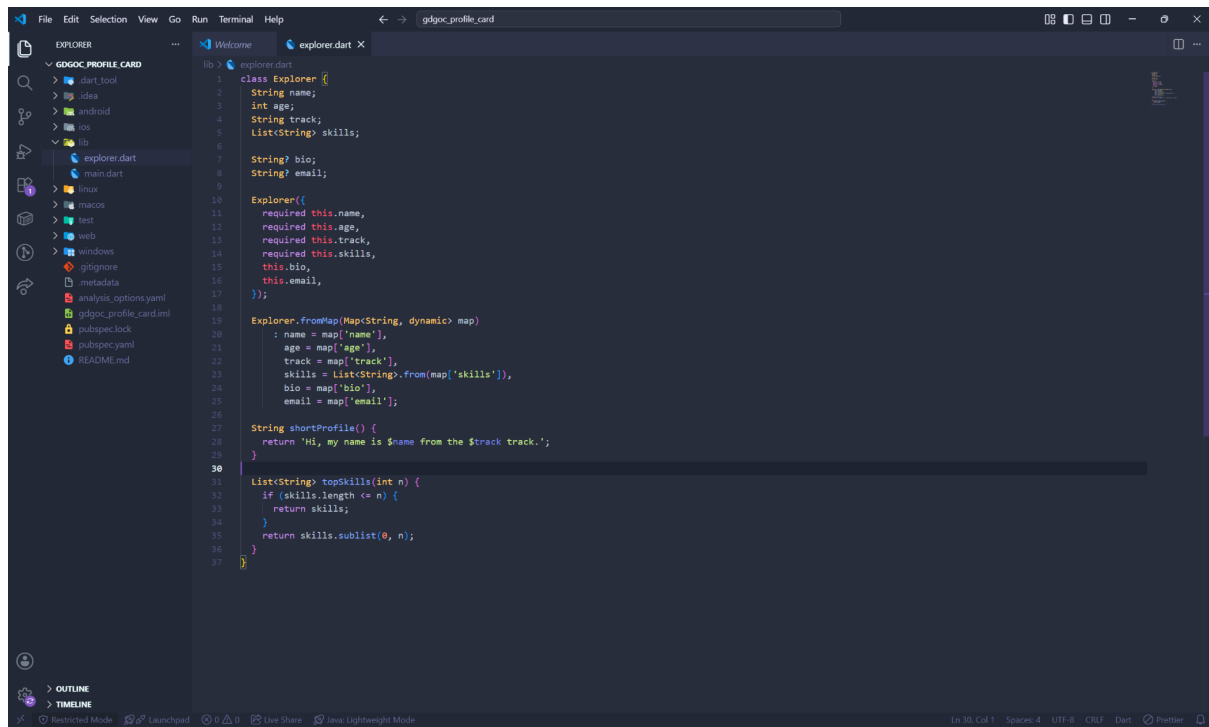
Your application code is in C:\gdgoc_profile_card\lib\main.dart.

C:\>
```

2. Create class model

Next, I created the data model for the application:

- I created a new file named `explorer.dart` inside the `lib/` folder.
- I defined the `Explorer` class, consisting of non-nullable fields (`name`, `age`, `track`, `skills`) and nullable fields (`bio`, `email`).
- I also added a named constructor `Explorer.fromMap` to handle data mapping, along with `shortProfile()` and `topSkills(int n)` methods as requested.

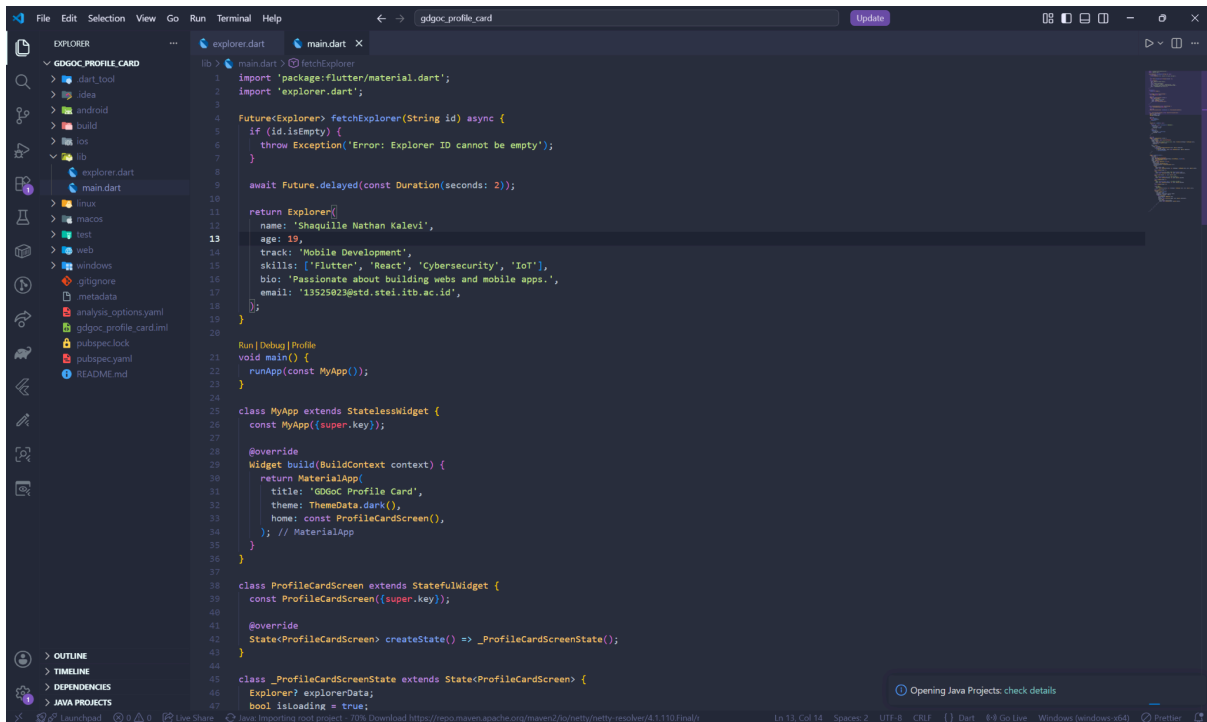
A screenshot of an IDE window showing the `explorer.dart` file. The file contains the `Explorer` class with fields `name`, `age`, `track`, `skills`, `bio`, and `email`. It includes a named constructor `Explorer.fromMap` and two methods: `shortProfile` and `topSkills`.

```
1 class Explorer {  
2   String name;  
3   int age;  
4   String track;  
5   List<String> skills;  
6  
7   String? bio;  
8   String? email;  
9  
10  Explorer({  
11    required this.name,  
12    required this.age,  
13    required this.track,  
14    required this.skills,  
15    this.bio,  
16    this.email,  
17  });  
18  
19  Explorer.fromMap(Map<String, dynamic> map)  
20  {  
21    name = map['name'];  
22    age = map['age'];  
23    track = map['track'];  
24    skills = List<String>.from(map['skills']);  
25    bio = map['bio'];  
26    email = map['email'];  
27  }  
28  
29  String shortProfile() {  
30    return 'Hi, my name is $name from the $track track.';  
31  }  
32  
33  List<String> topSkills(int n) {  
34    if (skills.length <= n) {  
35      return skills;  
36    }  
37    return skills.sublist(0, n);  
38  }  
39 }
```

3. Create async function

For the asynchronous logic, I simulated fetching user data from a remote server.

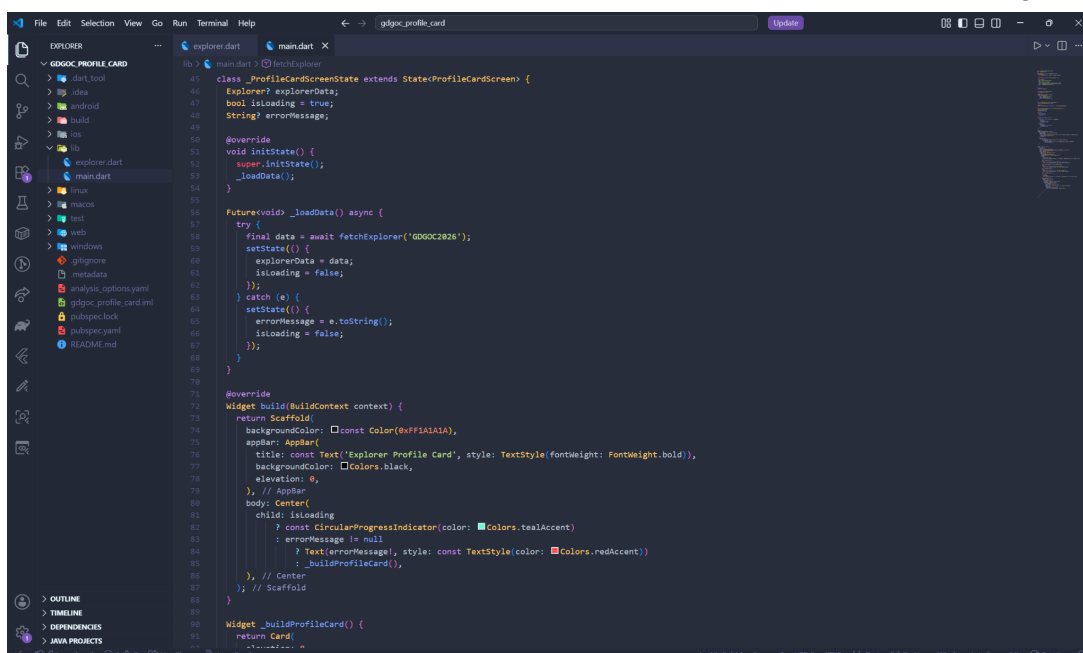
- I created a `Future<Explorer>` `fetchExplorer(String id)` function inside `main.dart`.
- I utilized `Future.delayed` to simulate a network delay of 2 seconds before returning the `Explorer` object.
- I added error handling by throwing an `Exception` if the provided `id` string is empty.



4. User Interface (UI)

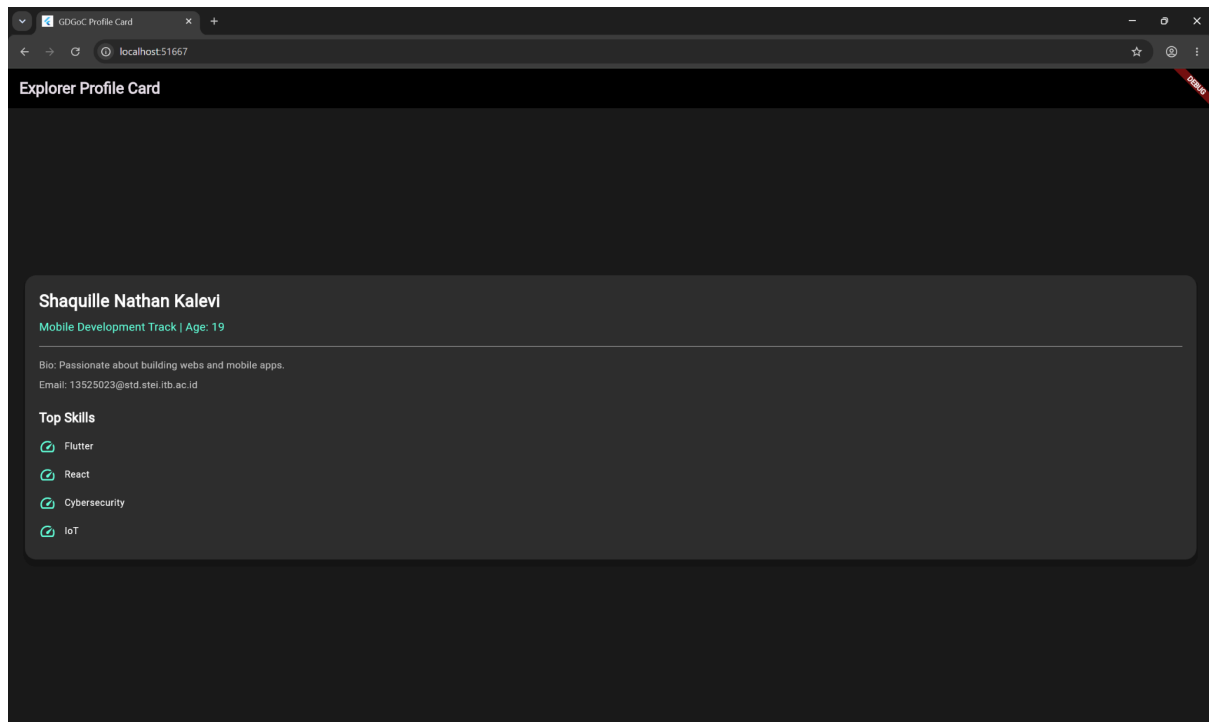
I combined the UI implementation and testing phase here.

- To gracefully handle errors, I wrapped the fetchExplorer() execution inside a try-catch block within the initState.
- I utilized Flutter's built-in widgets such as Card for the main layout container, Column to stack the texts vertically, and ListView.builder inside a Flexible widget to render the dynamic list of skills properly.
- For the nullable fields (bio and email), I used the null-aware operator ?? to display fallback texts like "No bio provided." in case the data is missing.



5. Testing

Finally, I ran the application via `flutter run*` to verify that the loading state, data mapping, and UI layout work as intended.

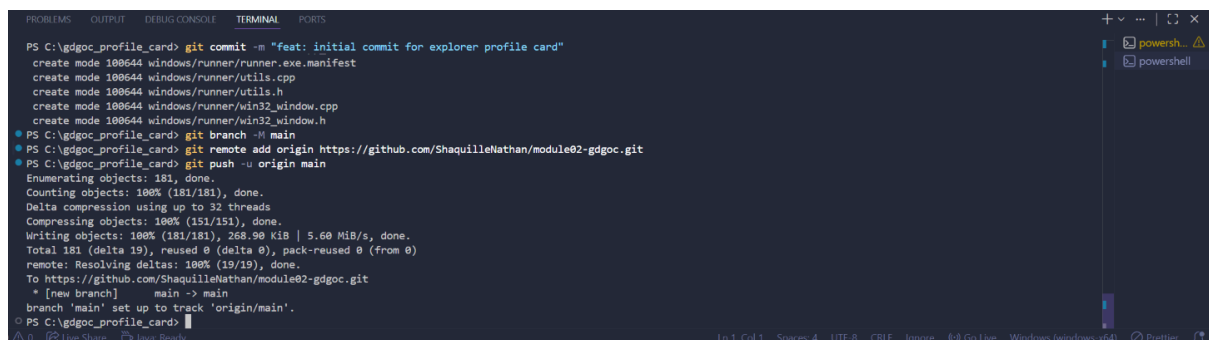


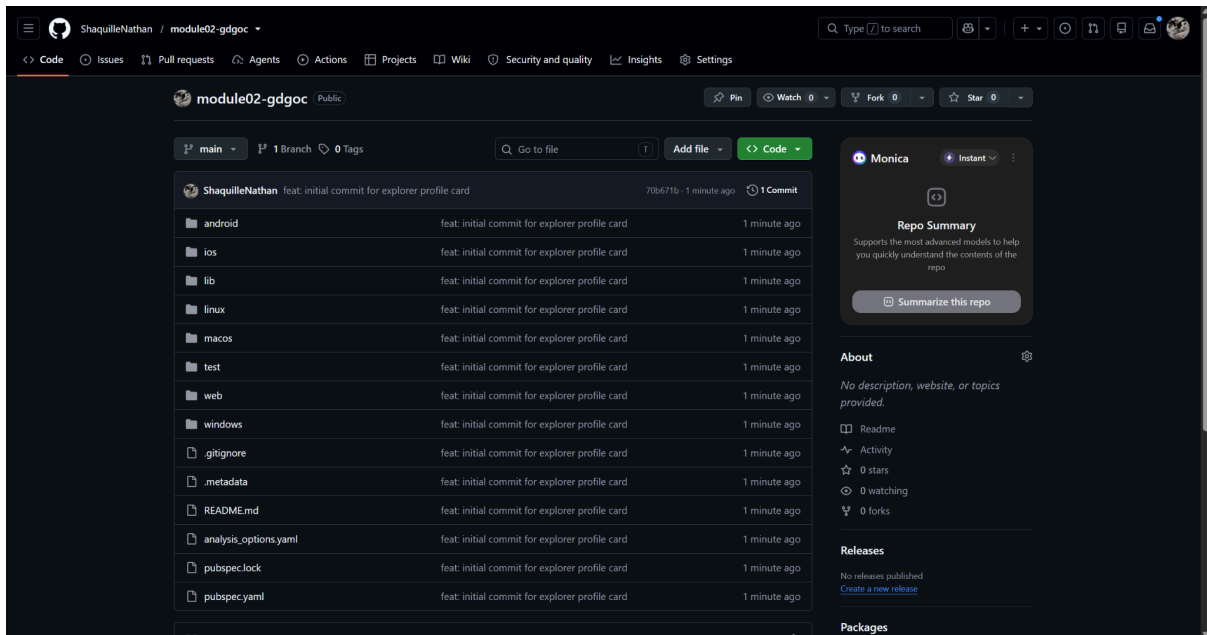
**intentionally, I want to test this on my physical device via USB cable like I did on Module 1, but somehow there's some problem with my usb cable. So, I test this on my google chrome desktop as the alternative.*

6. Github Initialization

For the final step, I pushed my local project to a public GitHub repository to maintain version control and repository hygiene.

- I verified the `.gitignore` file to ensure auto-generated directories like `build/` and `.dart_tool/` are not tracked.
- I initialized the git repository, committed my codes, and pushed it to GitHub.





You can see it here: <https://github.com/ShaquilleNathan/module02-gdgoc>

To-Do:

Expected Output

1. Screenshot of the running application. ✓
2. Screenshot of the code structure. ✓
3. Link to a public GitHub repository. ✓

Bonus Tasks

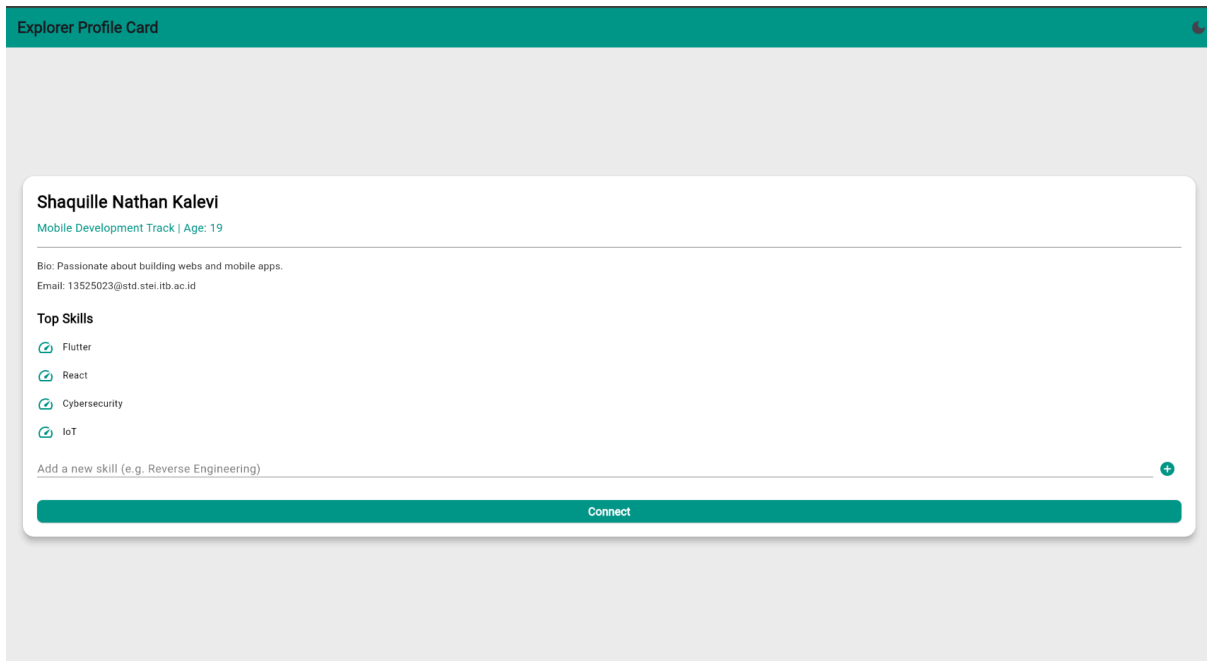
Bonus 1: Theme Toggle ⌚

Bonus 2: Dynamic Skill List with Add Button ⌚

Bonus 3: Async "Connect" Button with Loading State ⌚

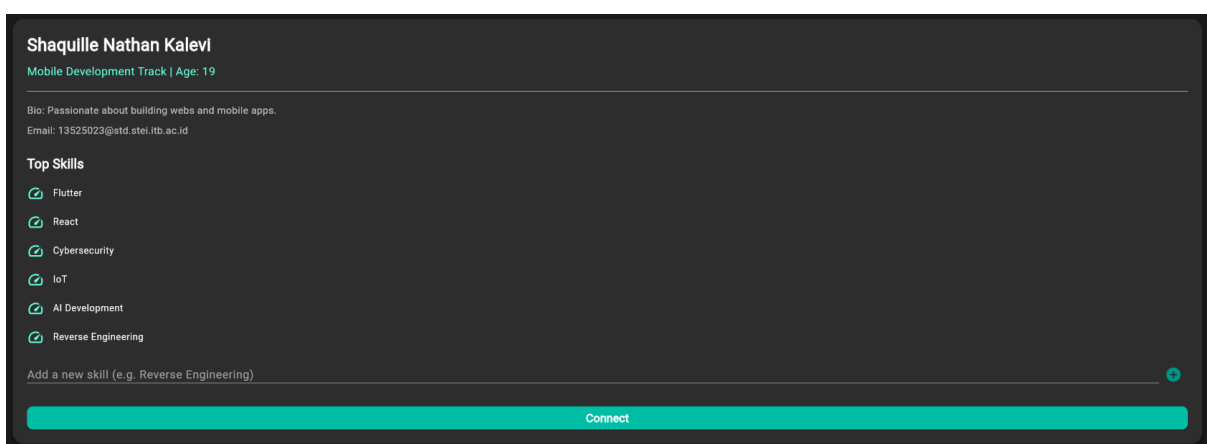
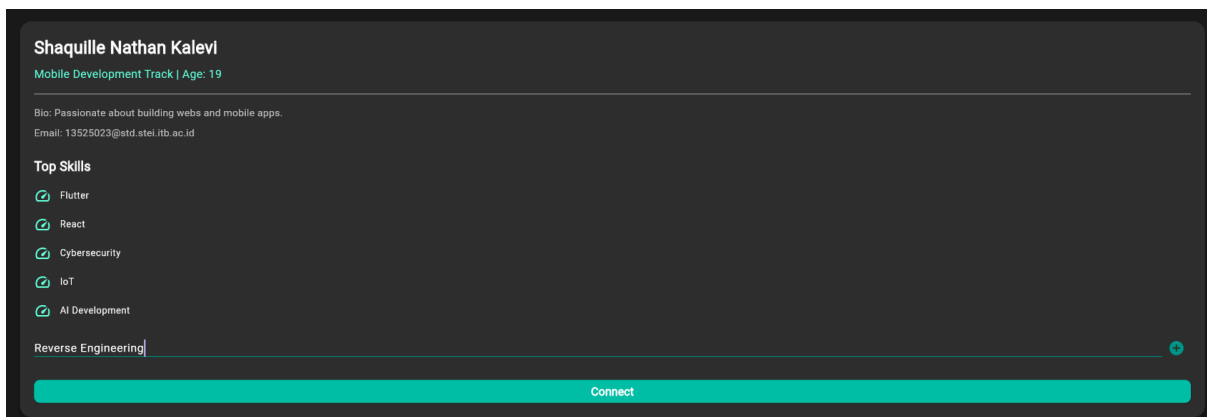
Bonus 1

To implement the theme toggle, I converted the main root application (MyApp) into a StatefulWidget to manage the isDarkMode boolean state. I placed an IconButton in the AppBar that triggers setState to switch between ThemeData.dark() and ThemeData.light().



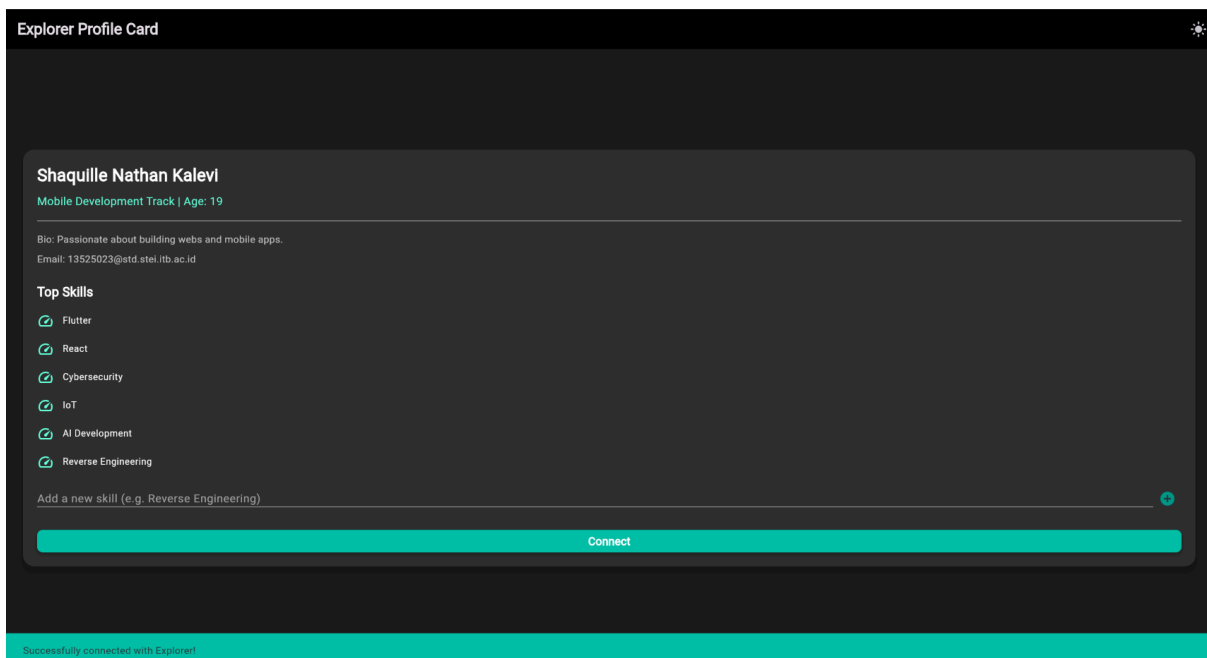
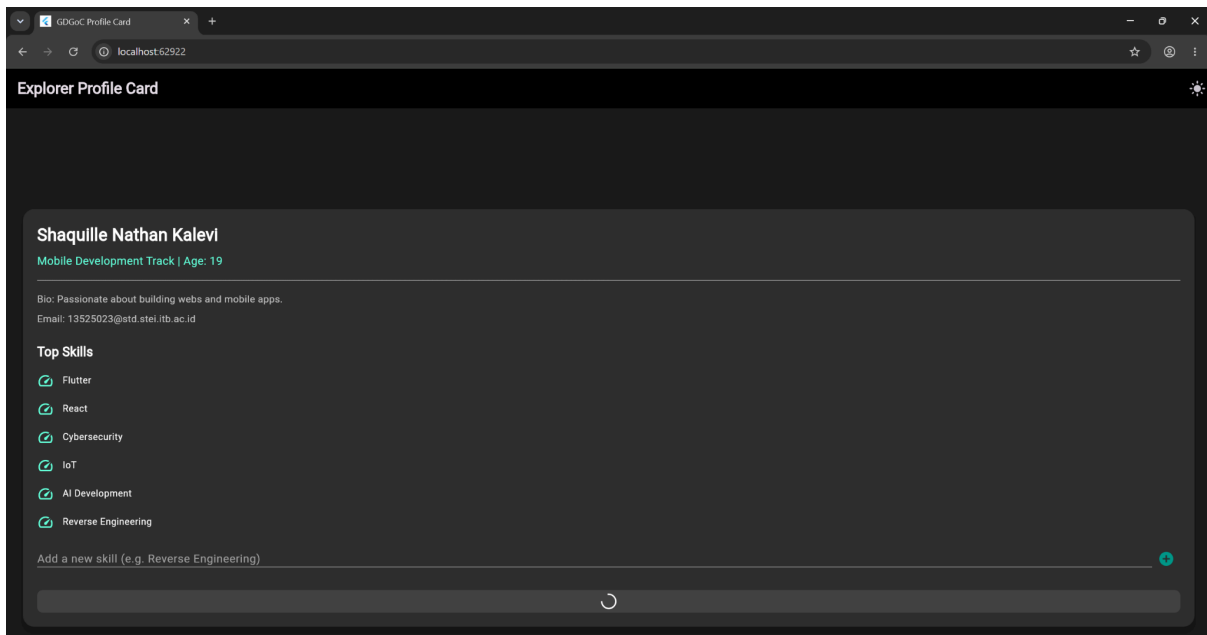
Bonus 2

I added a `TextField` utilizing a `TextEditingController` directly below the `ListView`. When the add button is pressed, the application invokes `setState` to push the inputted text into the `explorerData!.skills` array and clears the text field, immediately updating the rendered UI.



Bonus 3

I placed an `ElevatedButton` at the bottom of the card. Once pressed, it updates the `isConnecting` flag via `setState` to render a `CircularProgressIndicator` inside the button. After a simulated `Future.delayed` process finishes, it triggers `ScaffoldMessenger.of(context).showSnackBar()` to display a success message and resets the loading state.



To-Do:Expected Output

1. Screenshot of the running application. 
2. Screenshot of the code structure. 
3. Link to a public GitHub repository. 

Bonus Tasks

Bonus 1: Theme Toggle 

Bonus 2: Dynamic Skill List with Add Button 

Bonus 3: Async "Connect" Button with Loading State 